A Gentle Introduction to Operadic Consistency in AI Reasoning

William Betts

October 20, 2026

Abstract

A question can often be answered in several stages. A language model might answer the original question directly, or answer its subquestions and use those answers to finish the calculation. What should it mean for these procedures to agree? This article develops the mathematics needed to ask that question precisely. Starting with functions and composition, we introduce categories, functors, operads, typed operations, and algebras over operads. An arithmetic example then connects these ideas to question decomposition and consistency checks. We conclude by explaining how to read the empirical evidence, and why consistency, correctness, and faithful reasoning are different claims. The prerequisites are familiarity with sets, functions, and elementary algebra.

How to read this article

Keep one distinction in mind throughout: *building a composite question* and *asking a model to answer it* are different operations. This is an exposition of the June 2026 versions of the two papers [1, 2], with additional teaching examples and elementary arguments. All numerical model results are attributed to their authors; no new model experiments are reported here.

1 One question, several ways to answer it

Consider the question

$$Q = \text{What is twice the sum of 3 and 4?}$$

The correct answer is 14. We can obtain it in one calculation, or we can first compute $3 + 4 = 7$ and then double 7. These are two ways to evaluate the same mathematical expression:

$$2(3 + 4).$$

Now imagine giving these tasks to a language model m. Here $m(q)$ means the short answer the model returns when asked question q. For the moment, regard m as a fixed function; we return to randomness in Section 9. There are two procedures:

1. **Direct:** ask m the entire question Q.
2. **Decomposed:** ask m for the sum of 3 and 4, insert its answer into *What is twice* $\square$*?*, and ask m again.

A hypothetical model could answer 12 directly, but answer 7 and then 14 on the decomposed route. The disagreement is observable even without being told that 14 is correct. This makes disagreement a potentially useful diagnostic. It does not, by itself, identify which route failed.

The mathematical paper of Bottman and Richardson organizes question templates and their substitutions using an operad. It then asks for agreement across different partial compositions of a question tree [1, Sections 2.2–2.3]. We will reconstruct that idea through arithmetic before returning to natural language.

2 Composition: the first categorical idea

2.1 Functions are a model for arrows

If $f : A \to B$ and $g : B \to C$ are functions, their composite is

$$g \circ f : A \to C, \qquad (g \circ f)(a) = g(f(a)).$$

The output of f becomes the input of g. The intermediate set B matters: it tells us that these functions can be connected. A category abstracts this pattern. Its objects need not be sets, and its arrows need not be functions, but composition must remain well behaved.

Definition 2.1 (Category)**.** *A category $\mathcal{C}$ consists of objects; arrows $f : A \to B$ between objects; a composite $g \circ f : A \to C$ whenever the arrows fit; and an identity arrow $\mathrm{id}_A : A \to A$ for every object. These satisfy*

$$h \circ (g \circ f) = (h \circ g) \circ f, \qquad f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f.$$

The first condition is *associativity*: the order of grouping a chain of compositions does not change the resulting arrow. The second says that identity arrows do nothing. For example, **Set** is the category whose objects are sets and whose arrows are functions. The category of groups uses groups as objects and group homomorphisms as arrows. In both cases, the operation is ordinary function composition, but the permitted arrows differ.

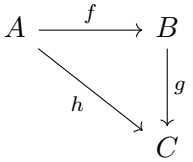
2.2 Preserving composition

Suppose we describe a procedure in one language and execute it in another. We want translating a composite procedure to agree with translating its pieces and composing the translations.

Definition 2.2 (Functor)**.** *A functor $F : \mathcal{C} \to \mathcal{D}$ assigns objects to objects and arrows to arrows, respecting their sources and targets, such that*

$$F(\mathrm{id}_A) = \mathrm{id}_{F(A)}, \qquad F(g \circ f) = F(g) \circ F(f).$$

The second equation expresses *preservation of composition*. One can read it as a comparison of two routes: compose first and translate afterward, or translate first and compose afterward. A *commutative diagram* is a diagram in which the composites along the relevant routes agree. For example,



commutes exactly when $h = g \circ f$.

This is the pattern we will reuse. We have not shown that an arbitrary language model defines a functor. For question templates with several input slots, the precise compatibility condition below will be the condition for an *algebra over an operad*.

3 Operads: composition with several input slots

3.1 Why keep the slots visible?

An ordinary arrow has one source object and one target object. A function of two real variables can already be represented categorically as $f : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$. Categories therefore can describe functions with many arguments. Operads make the individual argument positions explicit, so that inserting an operation into a particular slot is part of the structure. For example, let

$$f(x, y) = x + y, \qquad g(u, v) = uv.$$

Inserting g into the first slot of f gives

$$(f \circ_1 g)(u, v, y) = f(g(u, v), y) = uv + y.$$

Inserting g into the second slot gives a different operation:

$$(f \circ_2 g)(x, u, v) = f(x, g(u, v)) = x + uv.$$

The subscript records the slot. The order of the remaining inputs is retained.

Definition 3.1 (Arity and partial composition)**.** *The* arity *of an operation is its number of inputs. An operad $\mathcal{O}$ has a set $\mathcal{O}(k)$ of k-input operations for every $k \geq 0$, together with substitution maps*

$$\circ_i : \mathcal{O}(k) \times \mathcal{O}(\ell) \longrightarrow \mathcal{O}(k + \ell - 1), \qquad 1 \leq i \leq k.$$

These maps satisfy the two compatibility laws described below.

Why $k + \ell - 1$? One of the original k slots is replaced by an operation with ℓ slots:

$$\underbrace{k - 1}_{\text{old slots still open}} + \underbrace{\ell}_{\text{new slots}} = k + \ell - 1.$$

A zero-input, or *nullary*, operation supplies a value without asking for any further inputs. Formally, a map $\mathbb{R}^0 \to \mathbb{R}$ selects one real number because $\mathbb{R}^0$ is a singleton. Substituting such an operation closes one slot.

3.2 The two compatibility laws

There are two ways to continue a substitution: go deeper inside a newly inserted operation, or insert something into another slot.

Nested substitutions. For $f \in \mathcal{O}(k)$, $g \in \mathcal{O}(\ell)$, and $h \in \mathcal{O}(m)$, the expression

$$f(\ldots, g(\ldots, h(\ldots), \ldots), \ldots)$$

should not depend on which pair of operations we assemble first. Formally,

$$f \circ_i (g \circ_j h) = (f \circ_i g) \circ_{i+j-1} h. \tag{1}$$

The index $i + j - 1$ is the new location of the j-th slot of g.

Substitutions in separate slots. If $i < j$, inserting g into slot i and h into slot j should give the same composite whichever insertion happens first:

$$(f \circ_i g) \circ_{j+\ell-1} h = (f \circ_j h) \circ_i g. \tag{2}$$

The shift by $\ell - 1$ accounts for the number of slots added by g. For a concrete example, take

$$f(x, y) = x - y, \quad g(a, b) = a + b, \quad h(c) = 2c.$$

Both sides of (2), with $i = 1$ and $j = 2$, yield

$$(a + b) - 2c.$$

This remains true even though subtraction itself is not an associative binary operation. Operadic associativity concerns *substitution of operations*, not a requirement that every binary operation satisfy $f(f(x, y), z) = f(x, f(y, z))$.

A convention worth keeping straight

We use ordered input slots and do not require identity operations. This is the non-symmetric, non-unital convention used in the mathematical paper [1, Section 2]. Non-unital does not forbid zero-input operations: an identity has one input, while a constant has none. With identities included, a colored operad having only unary operations is a category.

4 Question templates and types

A question template has blanks to be filled. Consider

$$a = \textit{What is the sum of } \square_1 \textit{ and } \square_2\textit{?}, \qquad d = \textit{What is twice } \square\textit{?}.$$

These have two and one input slots, respectively. The composite $d \circ_1 a$ is a template for asking for twice a sum. It is useful to distinguish a template from a fully specified question:

$$\begin{aligned} a &= \textit{What is the sum of } \square_1 \textit{ and } \square_2\textit{?} && \text{has two blanks,} \\ s &= \textit{What is the sum of 3 and 4?} && \text{has no blanks.} \end{aligned}$$

The word *input* refers here to an unfilled argument slot. It does not refer to every piece of information appearing in the sentence.

4.1 Colors are types

Real questions often take and return different kinds of values. The template *What is the birth date of* $\square$*?* takes a person and returns a date. The template *How old was* $\square_1$ *on* $\square_2$*?* could take a person and a date and return an age in years. A *colored operad* attaches such types to the slots and outputs:

$$\mathcal{O}(c_1, \ldots, c_k; d)$$

is the collection of operations with input types $c_1, \ldots, c_k$ and output type d. For example,

$$\mathsf{birthdate} \in \mathcal{O}(\mathsf{Person}; \mathsf{Date}), \qquad \mathsf{age} \in \mathcal{O}(\mathsf{Person}, \mathsf{Date}; \mathsf{Years}).$$

A substitution is permitted only when the inserted operation's output type matches the receiving slot's input type. For our arithmetic example,

$$s \in \mathcal{Q}(;\mathsf{Number}), \qquad d \in \mathcal{Q}(\mathsf{Number};\mathsf{Number}), \qquad d \circ_1 s \in \mathcal{Q}(;\mathsf{Number}).$$

Type checking is useful but limited. It can reject inserting a person's name into a slot expecting a number. It cannot determine whether a supplied number is factually correct. Even numerical types may need refinement: a distance in miles and a duration in minutes should not be silently treated as interchangeable.

4.2 Formal substitution and natural language

In a formal expression, substitution is exact:

$$\mathsf{double}(\mathsf{sum}(3,4)).$$

A natural-language rendering could be *What is twice the sum of 3 and 4?* A different rendering might use different words while expressing the same calculation. This suggests a careful mathematical approach: define substitutions on structured expressions first, then separately specify how to render those expressions as prompts. If one instead identifies expressions by meaning, the chosen equivalence must respect substitution. In other words, replacing a subexpression by an equivalent one must preserve the meaning of the whole. This requirement is called *congruence*. Natural language makes that requirement substantive. A rewrite can change scope, omit a condition, or change what a pronoun refers to. In that case, different answers might reflect different questions. A trustworthy consistency check needs a decomposition that preserves the original task.

5 Algebras: giving abstract operations a meaning

An operad specifies how operations fit together. An *algebra over the operad* specifies how those operations act on actual values. For each type c, choose a set $A(c)$ of values of that type. For each operation $q \in \mathcal{O}(c_1, \ldots, c_k; d)$, choose a function

$$\varphi_q : A(c_1) \times \cdots \times A(c_k) \longrightarrow A(d).$$

For a numerical interpretation, $A(\mathsf{Number})$ might be $\mathbb{R}$. The sum template would act by addition, and the doubling template would act by multiplication by two. The assignment must preserve substitution:

$$\boxed{\varphi_{q \circ_i r} = \varphi_q \circ_i \varphi_r.} \tag{3}$$

The left side interprets the composite operation. The right side composes the interpretations of its parts. This is the many-input counterpart of the composition-preservation equation for a functor. For example, if $\varphi_a(x, y) = x + y$ and $\varphi_d(x) = 2x$, compatibility requires

$$\varphi_{d \circ_1 a}(x, y) = 2(x + y).$$

One cannot independently assign $\varphi_{d \circ_1 a}(x, y) = 2x + y$ and still claim to have an algebra with those interpretations of a and d.

5.1 The model supplies a candidate interpretation

Given a model m, define a candidate interpretation of a template by filling its blanks with values and querying the model:

$$\varphi_q^m(v_1, \ldots, v_k) = m(\text{the prompt obtained from } q \text{ using } v_1, \ldots, v_k).$$

This assumes that the response can be parsed into the expected value type. An unparseable or ill-typed response needs an explicit failure convention. For the fully specified sum question s, φ_s^m is simply its answer. The compatibility condition becomes

$$m(d \circ_1 s) = m(d[m(s)]), \tag{4}$$

where $d[v]$ means the doubling template with the value v inserted.

An arbitrary model is not automatically an algebra

Assigning a function to every template supplies candidate algebra data. To obtain a genuine algebra, those functions must satisfy (3). Testing agreement on selected questions checks selected instances of that law; passing finitely many tests does not establish the law for all operations.

The distinction between substitution and evaluation is now visible. The expression $d \circ_1 s$ is constructed from two *questions*. The expression $d[m(s)]$ contains an *answer* returned by the model. The first route asks the whole question; the second asks its parts.

6 Trees and partial collapses

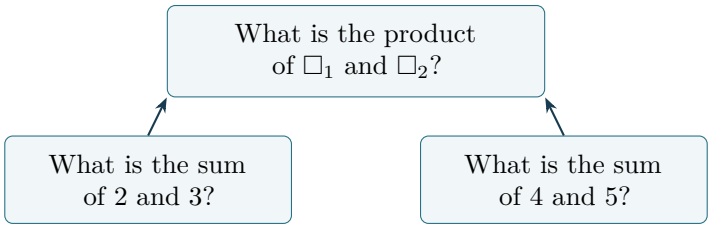
A chain is enough for the doubling example. A tree becomes useful when a question needs several intermediate answers. Consider

$$Q' = \textit{What is the product of the sum of 2 and 3 and the sum of 4 and 5?}$$

Its intended arithmetic expression is

$$(2 + 3)(4 + 5) = 45.$$

The decomposition has two subquestions feeding one final template:



Answers flow from the leaves toward the root.

To *collapse* an internal edge is to combine the question templates at its ends into one question. It is a change in how the computation is packaged. For this tree there are four choices:

Collapsed edges	Procedure
Neither	Ask both sum questions, then ask for the product of their answers.
Left only	Ask for the sum of 4 and 5, then ask for $(2 + 3)$ times that answer.
Right only	Ask for the sum of 2 and 3, then ask for that answer times $(4 + 5)$.
Both	Ask the original question about $(2 + 3)(4 + 5)$ directly.

All four procedures should return 45 under exact arithmetic. A model might give different answers. The theory's full consistency requirement compares all partial collapses, including the original tree and its total collapse [1, Section 2.3].

Definition 6.1 (Operadic consistency on a tree)**.** *Write* $\mathrm{ev}_m(T')$ *for the final answer obtained by evaluating a question tree* T' *from its leaves to its root. A model* m *is* operadically consistent on T *if*

$$\mathrm{ev}_m(T'_1) = \mathrm{ev}_m(T'_2) \quad \textit{for all partial collapses } T'_1, T'_2 \textit{ of } T.$$

Here equality is equality of the interpreted answer values.

For a tree with n operation vertices there are $n - 1$ internal edges, so there are 2^{n-1} subsets of edges to collapse. This counts choices of contraction; different choices can occasionally give identical rendered questions. Checking every choice can become expensive.

Proposition 6.2 (Exact interpretation is independent of collapse)**.** *Suppose that the assigned functions form an algebra over an operad. Then evaluating a finite operation tree gives the same result after any sequence of internal-edge collapses.*

Proof. Consider one edge connecting an operation r to slot i of an operation q. Before collapsing, the corresponding local computation uses $\varphi_q \circ_i \varphi_r$. After collapsing, it uses $\varphi_{q \circ_i r}$. These agree by (3). Therefore the value supplied to the rest of the tree does not change. Repeating this argument for each collapsed edge proves the claim. □

This elementary argument explains the diagnostic: a disagreement witnesses a failure of the tested compatibility condition, assuming fixed prompts, valid decomposition, and a suitable rule for comparing answers. There is also a distinction between *agreement at the two endpoints* and *agreement across every partial collapse*. On a large tree, the fully decomposed and fully collapsed computations could agree while an intermediate collapse disagrees. For a two-vertex chain, there is just one internal edge, so these two requirements coincide.

7 A practical consistency check

For the two-question example, the computation can be written as follows:

Three-query procedure

1. $a_{\text{direct}} \leftarrow m(Q)$.
2. $b \leftarrow m(s)$.
3. $a_{\text{parts}} \leftarrow m(d[b])$.
4. Compare a_{direct} and a_{parts}.

For this arithmetic example, define

$$\text{OC}_{\text{toy}}(m, Q) = \begin{cases} 1, & \text{if the two final answers represent the same number,} \\ 0, & \text{otherwise.} \end{cases}$$

A number parser should recognize that *14*, *fourteen*, and *The answer is 14* can give the same numerical answer. If parsing fails, one should record that failure rather than silently treating it as a valid number. More generally, a comparison function S can assign partial credit:

$$\text{OC}(m, Q) = S(a_{\text{direct}}, a_{\text{parts}}) \in [0, 1].$$

The authors' implementation uses task-specific comparisons of direct and decomposed outputs; its cached records also contain reference answers for evaluating correctness [3]. A numerical score is an operational approximation to agreement, not automatically a mathematical equivalence relation. For instance, a thresholded similarity measure need not be transitive.

7.1 Agreement and correctness are different variables

Let y be the correct answer. A consistency check compares two model outputs. A correctness check compares an output with y. The following are all logically possible; the entries are hypothetical, and the correct answer throughout is 14:

Direct	Decomposed	Agree?	Direct answer correct?
14	14	Yes	Yes
12	12	Yes	No
14	12	No	Yes
12	14	No	No

Consequently, neither

$$\text{consistent} \implies \text{correct} \quad \text{nor} \quad \text{inconsistent} \implies \text{direct answer wrong}$$

is a valid implication. The empirical question is whether consistency helps predict correctness on the tasks being studied. This also explains the phrase *label-free signal*. Computing agreement does not require the correct answer. Measuring how useful agreement is as a predictor does require

independently assessed correctness. A later calibration step may use labeled examples even though the raw signal does not.

A useful comparison is *chain-of-thought self-consistency*. A generated chain of thought is text presenting intermediate reasoning steps. Self-consistency samples several such responses to the same whole question and compares their final answers. OC changes the structure of the queries by using a prescribed decomposition.

8 What the experiments say, and how to read them

Bottman, Liu, and Richardson test two-step chains on twelve instruction-tuned models across HotpotQA, MuSiQue, StrategyQA, and DROP. They report cross-model correlations between consistency rate and accuracy of $0.92, 0.94, 0.94, 0.86$, respectively. Their selective-prediction comparison adds OC to a chain-of-thought self-consistency predictor using $K = 3$ sampled responses. Reported gains are 0.086–0.096 in AUARC and 0.092–0.164 in AUROC. These are ranking-metric gains, not increases in the accuracy of every generated answer. The paper also studies five thinking models, extracting decompositions from their reasoning traces [2, Sections 3.1–3.2; Appendix A.13].

Three distinctions help interpret this evidence.

8.1 A correlation across models is not a confidence level

Imagine plotting one point per model, with its consistency rate on the horizontal axis and its accuracy on the vertical axis. A strong positive correlation says that models with higher consistency rates tend to have higher accuracy in that comparison. It does not say that an answer with consistency score 0.94 has a 94% probability of being right. Those are different statistical objects. To investigate confidence for individual answers, one needs question-level evaluation and, for probability estimates, calibration.

8.2 Selecting answers is different from fixing them

A system can rank its answers by a confidence signal and return only the highest-ranked ones. The fraction it returns is its *coverage*. The fraction correct among the returned answers is its *selective accuracy*. For illustration, suppose a model answers 80 of 100 questions correctly. A filter might retain 50 answers, of which 45 are correct. Its coverage is 50%, and its selective accuracy is 90%. The underlying collection of answers still contains 80 correct answers; the filter has changed which answers are retained. An area-under-curve metric summarizes performance over many thresholds. AUROC is the area under a receiver operating characteristic curve; here it measures how well correct answers rank above incorrect answers, giving half credit to ties. The authors' AUARC metric summarizes selective accuracy as the retained fraction varies [2, Appendix A.12]. Higher values reward a ranking that puts correct answers near the top across different choices of coverage.

8.3 A budget label is not a complete cost accounting

The reported combined predictor uses both OC and self-consistency. The protocols describe three queries for OC and K sampled chains plus answer-extraction calls for self-consistency [2, Section 3.1; Appendix A.3–A.4]. This creates a practical question for the reader: which calls are actually shared when both features are used? Matching the cost of each signal separately does not establish the total cost of their combination. Prompt length, generated tokens, decomposition generation,

and auxiliary models also matter. The ranking improvement can be useful evidence while the total deployment cost still needs its own accounting.

9 What category theory contributes

The elementary check is easy to describe without category theory: compare a direct answer with a decomposed answer. The mathematics provides a systematic account of *which compositions are being compared* and *what law those comparisons test*. First, it separates structure from interpretation. The same tree can be interpreted by exact arithmetic, by a language model, or by another computational system. The tree describes the arrangement of operations; the interpretation supplies their behavior. Second, it turns a local equality into a general organizing principle. Equation (3) explains why every partial collapse should agree under an exact interpretation. A tree with several branches is then a natural extension of a two-step chain. Third, it exposes assumptions that can otherwise remain implicit. We need compatible types, a valid decomposition, and an answer comparison suited to the task. Each is an independent place where a proposed check can fail.

9.1 What still needs care

Shared mistakes. Both routes can inherit the same incorrect fact or arithmetic rule. Agreement does not supply external evidence for the answer.

A changed task. An invalid decomposition, missing premise, or misleading rewrite can create disagreement even when a model answers each displayed prompt appropriately.

Randomness and context. Our mathematical model treated m as a function. Actual evaluations must specify prompts, decoding settings, available evidence, and context handling. With stochastic outputs, equality of individual samples and equality of output distributions are different requirements. Repeated sampling can help study that distinction, at additional cost.

Behavior versus internal reasoning. Agreement between prompted computations is evidence about the model's behavior under those prompts. It does not establish that the model used the displayed decomposition internally when answering the original question. Even an explanation extracted from generated reasoning is an object to be evaluated, rather than automatic access to the computation that caused the answer.

The useful mathematical question is therefore specific: does the chosen interpretation respect the chosen composition on the questions we test? The useful empirical question is separate: how much does knowing the answer to that question help us decide which model outputs to trust?

10 A route back to the papers

With the main ideas in place, the original notation becomes easier to navigate:

Reading	Question to keep beside it
Theory paper, §1	What changes when we answer a question in stages?
§2.1	Can I compute the arity of a partial composition and explain both associativity laws?
§2.1.1	What makes an interpretation an algebra rather than an arbitrary assignment of functions?
§2.2	Which objects are question templates, and which objects are answers?
§2.3	Which partial collapses are compared? How many are possible?
Empirical paper, §3; A.3 and A.8	What was queried, what was scored, and when can agreement mislead us?

A useful first reading can postpone the loop-space and cohomology discussions. They provide wider mathematical context, but the substitution laws already explain the central diagnostic. The public repository separates cached signals from the scripts that run model inference. Its analysis can therefore be inspected without generating new model responses [3]. Reproducing those analyses would be a separate project; understanding the definitions does not require it.

References

[1] Nathaniel Bottman and Kyle Richardson. *Operads for compositional reasoning in LLMs*. arXiv:2606.13634, version 1, June 11, 2026. https://arxiv.org/abs/2606.13634v1.

[2] Nathaniel Bottman, Yinhong Liu, and Kyle Richardson. *Operadic consistency: a label-free signal for compositional reasoning failures in LLMs*. arXiv:2606.13649, version 1, June 11, 2026. https://arxiv.org/abs/2606.13649v1.

[3] Nathaniel Bottman and collaborators. *Operadic consistency: companion code and cached data*. GitHub repository. Accessed October 20, 2026. https://github.com/natebottman/operadic-consistency-paper.